

AWS EC2 배포 기초

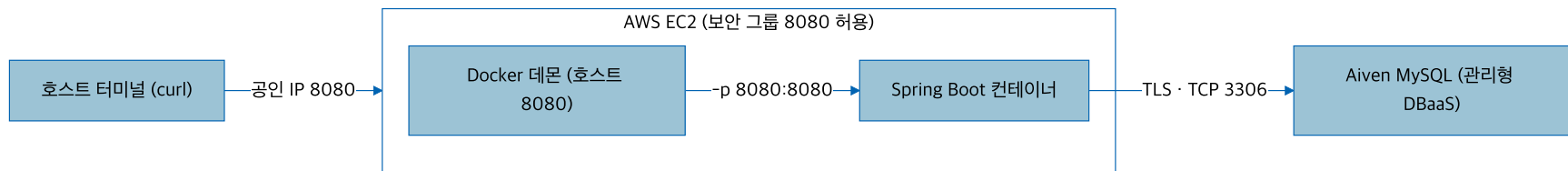
학습 목표

- 중지된 EC2 인스턴스의 유형을 `t4g.micro` 로 변경해 재시작하고 갱신된 공인 IP를 확보합니다
- SSH로 원격 Ubuntu 서버에 접속해 CPU 아키텍처, 메모리, 디스크와 사설·공인 IP 및 리스닝 포트를 점검합니다
- 공식 설치 스크립트로 EC2에 Docker 엔진을 프로비저닝하고 데몬 상태와 `aarch64` 호환성을 검증합니다
- GitHub Actions에 `linux/arm64` 멀티 플랫폼 타깃을 추가해 GHCR이미지를 빌드하고 EC2에서 다운로드합니다
- 외부 관리형 DB `Aiven MySQL` 접속 정보를 파일로 주입해 컨테이너를 구동하고 외부 응답을 검증합니다

IaaS 배포는 가상 서버에 런타임을 직접 설치하는 것에서 시작합니다. 본 차시는 인스턴스 프로비저닝, 컨테이너 런타임 설치, 외부 관리형 DB 연동을 하나의 통신 경로로 연결해 완결합니다.

핵심 개념 및 구조

단일 컨테이너와 외부 관리형 DB 연동 배포 구조



🔒 보안 그룹

인스턴스 단위 방화벽이 인바운드 8080을 허용해야 외부 요청이 도달합니다

➡ 포트 매핑

호스트 8080으로 들어온 패킷을 컨테이너 내부 8080으로 전달합니다

📄 외부 DB 연동

애플리케이션이 인터넷 구간으로 관리형 MySQL에 직접 접속합니다

외부 요청은 **보안 그룹** → **호스트 8080** → **컨테이너 8080** 순서로 도달합니다. 데이터베이스는 VPC 내부가 아니라 인터넷 구간 너머에 있으므로 아웃바운드 통신과 TLS 암호화가 전제됩니다.

배포 모델 비교: PaaS와 IaaS의 책임 경계

구분	PaaS 배포 (Render, Railway)	IaaS 배포 (AWS EC2)
컴퓨팅 제어권	플랫폼이 서버와 OS를 은닉하고 런타임만 제공	OS, CPU 아키텍처, 커널 설정을 직접 통제
네트워크 격리	플랫폼 제공 기본 도메인과 포트에 의존	사설망, 보안 그룹, 포트 매핑 전권 보유
운영 책임	장애 조치와 패치를 벤더가 대행	엔진 설치, 데몬 관리, 프로세스 점검 직접 수행

IaaS에서는 런타임이 기본 제공되지 않습니다. 인스턴스에 Docker 엔진을 직접 설치하고 데몬을 상주시키는 프로비저닝 단계가 반드시 선행됩니다.

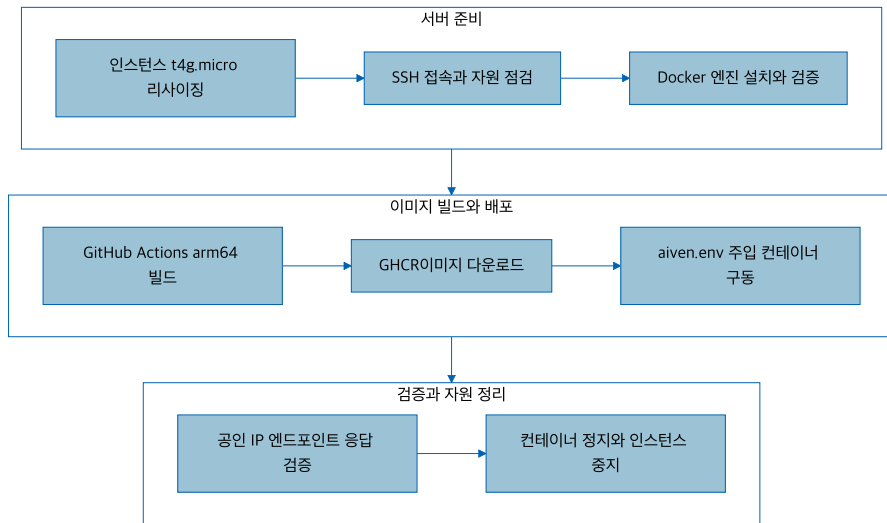
IP 주소 체계: 사설 IP · 공인 IP · 탄력적 IP

IP 주소 유형	정의	특징 및 용도
사설 IP (Private IP)	VPC 내부 통신 전용 주소 (172.31.x.x)	인터넷에서 직접 접근 불가, 동일 VPC 내부 통신
공인 IP (Public IP)	인터넷 게이트웨이를 경유하는 외부 통신 주소	인스턴스 중지 후 재시작 시 동적으로 재할당
탄력적 IP (Elastic IP)	계정에 영구 고정 할당되는 공인 IPv4	재시작에도 불변 유지, 도메인 DNS 연동에 필수

탄력적 IP를 연결하지 않았으므로 인스턴스를 재시작할 때마다 공인 IP가 바뀝니다. 접속 주소는 매번 다시 조회해 환경변수에 담아야 합니다.

실습 단계

실습 전체 흐름: 인스턴스 리사이징부터 자원 중지까지



구간	검증 지점
서버 준비	<code>uname -m</code> 이 <code>aarch64</code> , <code>free -h</code> 가 약 1GiB, <code>ss -tulnp</code> 에 8080 미개방
배포와 검증	GHCR 태그에 <code>linux/amd64</code> · <code>linux/arm64</code> 동시 표기, 공인 IP 8080 응답 <code>200 OK</code>

인스턴스 유형 변경과 재시작

```
export MY_KEY_NAME="student01-key"
export AWS_PROFILE="student01"
export AWS_REGION="ap-northeast-2"
export AWS_PAGER=""

export INSTANCE_ID=$(aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=student01-test-ec2" \
    "Name=instance-state-name,Values=stopped" \
  --query "Reservations[0].Instances[0].InstanceId" --output text)

aws ec2 modify-instance-attribute \
  --instance-id "$INSTANCE_ID" \
  --instance-type "Value=t4g.micro"
aws ec2 start-instances --instance-ids "$INSTANCE_ID"
aws ec2 wait instance-status-ok --instance-ids "$INSTANCE_ID"

export PUBLIC_IP=$(aws ec2 describe-instances \
  --instance-ids "$INSTANCE_ID" \
  --query "Reservations[0].Instances[0].PublicIpAddress" \
  --output text)
echo "새로 할당된 공인 IP: $PUBLIC_IP"
```

명령	설명
<code>modify-instance-attribute</code>	<code>stopped</code> 상태에서만 인스턴스 유형 변경 가능
<code>wait instance-status-ok</code>	시스템·인스턴스 상태 검사 2/2 통과까지 대기
<code>PublicIpAddress</code> 조회	재시작으로 갱신된 접속 IP를 변수에 자동 할당

`t4g.nano`의 512MiB 메모리로는 JVM 힙과 Docker 데몬이 공존하기 어려워 `t4g.micro` (1GiB)로 상향합니다.

SSH 원격 접속과 시스템 자원 점검

```
ssh -i ~/.ssh/"$MY_KEY_NAME".pem \  
-o StrictHostKeyChecking=accept-new \  
ubuntu@"$PUBLIC_IP"
```

— 원격 EC2 터미널 내부 실행 —

```
whoami
```

```
uname -m
```

```
cat /etc/os-release | grep "PRETTY_NAME"
```

```
df -h /
```

```
free -h
```

명령

점검 포인트

`uname -m`

`aarch64` (ARM Graviton2) 정상 인식 확인

`free -h`

`t4g.micro` 수준의 약 1GiB 메모리 확인

`df -h /`

8GB EBS 루트 볼륨의 여유 공간 확인

`StrictHostKeyChecking=accept-new`는 처음 보는 호스트 키를 자동 등록합니다. 재시작으로 IP가 바뀐 서버에 접속할 때 대화형 확인 절차를 생략합니다.

사설 IP와 공인 IP 대조 및 리스닝 포트 기준선

```
# — 원격 EC2 터미널 내부 실행 —  
ip addr show | grep "inet "  
curl -s ifconfig.me  
echo ""  
  
sudo ss -tulnp  
  
exit
```

명령	확인 내용
<code>ip addr show</code>	VPC 대역 <code>172.31.x.x</code> 사설 인터페이스 주소
<code>curl ifconfig.me</code>	외부에서 관측되는 공인 IP, <code>\$PUBLIC_IP</code> 와 일치
<code>ss -tulnp</code>	22번 SSH만 열려 있고 8080은 아직 비어 있음

서버 내부의 네트워크 인터페이스에는 **공인 IP가 직접 부여되지 않습니다**. 인터넷 게이트웨이가 사설 IP와 공인 IP를 1:1로 변환해 중계합니다.

Docker 엔진 자동 설치 스크립트 실행

```
ssh -i ~/.ssh/"$MY_KEY_NAME".pem ubuntu@"$PUBLIC_IP"

# — 원격 EC2 터미널 내부 실행 —
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

sudo systemctl status docker --no-pager
```

구성 요소	설명
<code>get-docker.sh</code>	배포판을 자동 감지해 공식 APT 저장소 등록과 패키지 설치를 일괄 수행
<code>systemctl status docker</code>	데몬이 <code>Active: active (running)</code> 상태인지 확인

설치 스크립트는 `docker-ce`, `containerd`, `docker-buildx-plugin`, `docker-compose-plugin`을 함께 설치합니다. 운영 환경에서는 스크립트 대신 저장소를 명시적으로 등록해 버전을 고정합니다.

Docker 데몬 상태와 ARM64 아키텍처 검증

```
# — 원격 EC2 터미널 내부 실행 —  
sudo docker --version  
sudo docker info \  
| grep -E "Architecture|Operating System|Server Version"  
  
sudo docker images  
sudo docker ps -a  
  
exit
```

점검 항목	기대 값
Architecture	aarch64 — ARM64 전용 런타임 환경 확인
docker images	빈 목록 — 다운로드된 이미지가 없는 초기 상태
docker ps -a	빈 목록 — 실행·중지 컨테이너가 없는 상태

`docker info`의 `Architecture`는 호스트 커널의 아키텍처입니다. 이 값이 곧 레지스트리에서 어떤 아키텍처 레이어를 선택할지 결정하는 기준이 됩니다.

GitHub Actions 멀티 플랫폼 타킷 추가

```
# docker-publish.yml - 기존 워크플로에서 변경된 부분
jobs:
  build-and-push:
    steps:
      - uses: actions/checkout@v6
      - uses: docker/setup-buildx-action@v4
      - uses: docker/build-push-action@v7
      with:
        context: .
        push: true
        platforms: linux/amd64,linux/arm64
        tags: ${ env.REGISTRY }}/${ env.IMAGE_NAME }}:latest
```

워크플로 속성	설명
<code>docker/setup-buildx-action</code>	QEMU 기반 멀티 아키텍처 빌더 구성
<code>platforms</code>	두 아키텍처를 매니페스트 리스트로 묶어 게시

GHCR로그인과 이미지 이름 소문자 변환 단계는 기존 워크플로를 그대로 유지합니다.

x86 러너에서 만든 AMD64 전용 이미지를 ARM 인스턴스에서 실행하면 `exec format error`로 기동이 거부됩니다.

커밋 푸시와 GHCR 매니페스트 확인

```
cd ~/workspace/simple-back
```

```
git add .github/workflows/docker-publish.yml  
git commit -m "ci: add linux/arm64 target for EC2 t4g deployment"  
git push
```

QEMU에뮬레이션으로 ARM64 레이어를 빌드하므로 단일 아키텍처 빌드보다 소요 시간이 길어집니다.

Actions 탭 확인

`Build and Push Docker Image to GHCR` 워크플로가 성공으로 종료되는지 확인합니다

Packages 태그 확인

태그 상세의 `OS/Arch`에 `linux/amd64`와 `linux/arm64`가 함께 표기되는지 확인합니다

빌드 로그 확인

멀티 플랫폼 빌드 단계에서 두 아키텍처 레이어가 함께 생성되는지 확인합니다

GHCR이미지 다운로드와 환경변수 파일 작성

```
ssh -i ~/.ssh/"$MY_KEY_NAME".pem ubuntu@"$PUBLIC_IP"

# — 원격 EC2 터미널 내부 실행 —
export GITHUB_USERNAME="본인_깃허브_아이디"

sudo docker pull \
  ghcr.io/"$GITHUB_USERNAME"/simple-back:latest

cat <<'EOF' > aiven.env
SPRING_DATASOURCE_URL=jdbc:mysql://호스트:포트/defaultdb?sslMode=REQUIRED
SPRING_DATASOURCE_USERNAME=avnadmin
SPRING_DATASOURCE_PASSWORD=발급받은_Aiven_암호
APP_MESSAGE=Live on AWS EC2 t4g.micro via GHCR!
EOF
cat aiven.env
```

항목	설명
<code>docker pull</code>	호스트 커널 <code>aarch64</code> 에 맞는 arm64 레이어를 자동 선택해 다운로드
<code>aiven.env</code>	자격 증명을 파일로 분리해 명령 이력에 암호가 남지 않게 격리

접속 정보를 `docker run -e` 인자로 직접 넘기면 `~/.bash_history`와 `ps` 출력에 평문으로 노출됩니다. 12-Factor App 원칙에 따라 파일로 주입합니다.

외부 DB 연동 컨테이너 백그라운드 구동

— 원격 EC2 터미널 내부 실행 —

```
sudo docker run -d \  
  --name simple-back \  
  --restart unless-stopped \  
  -p 8080:8080 \  
  --env-file aiven.env \  
  ghcr.io/"$GITHUB_USERNAME"/simple-back:latest
```

```
sudo docker ps
```

```
sudo docker logs --tail 30 simple-back
```

```
sudo ss -tulnp | grep 8080
```

```
exit
```

실행 옵션

설명

`-d`

터미널을 점유하지 않고
백그라운드 데몬으로 상주

`--restart unless-
stopped`

호스트 재부팅 후 데몬 기동 시
컨테이너 자동 복구

`--env-file
aiven.env`

파일의 키·값을 컨테이너
환경변수로 일괄 주입

부팅 로그에서 `HikariPool` 커넥션 획득과 `Started Application` 메시지를 확인하면 외부 DB 연동까지 성립한 것입니다.

호스트 머신에서 외부 엔드포인트 응답 검증

```
# 로컬 호스트 터미널에서 실행
curl -i http://"$PUBLIC_IP":8080/
curl -i http://"$PUBLIC_IP":8080/users
```

엔드포인트	기대 응답
/	200 OK와 APP_MESSAGE 환경변수 값
/users	200 OK와 Aiven DB에서 조회한 JSON

원격 서버 내부의 `curl localhost:8080`이 아니라 **로컬 터미널에서 공인 IP로** 요청해야 인터넷 구간을 포함한 전체 경로가 검증됩니다.

✓ 응답 확인

상태 코드 `200 OK`와 함께 JSON 데이터가 반환되는지 확인합니다

📡 통신 경로

보안 그룹 → 호스트 포트 → 컨테이너 → 외부 Aiven DB로 이어지는 파이프라인이 완성됩니다

컨테이너 정지와 인스턴스 중지 비용 통제

```
ssh -i ~/.ssh/"$MY_KEY_NAME".pem ubuntu@"$PUBLIC_IP" \  
"sudo docker stop simple-back"  
  
aws ec2 stop-instances --instance-ids "$INSTANCE_ID"  
aws ec2 wait instance-stopped --instance-ids "$INSTANCE_ID"  
echo "인스턴스가 안전하게 중지되었습니다."
```

\$ 컴퓨팅 요금

인스턴스가 `running` 인 시간에만 부과되며 중지 즉시 0원이 됩니다

📦 스토리지 요금

EBS 볼륨은 중지 상태에서도 프로비저닝된 용량 기준으로 계속 과금됩니다

명령

설명

```
ssh ... "docker  
stop"
```

원격 셸에 진입하지 않고 단발 명령으로 컨테이너 정지

```
aws ec2 stop-  
instances
```

CPU·메모리 컴퓨팅 과금을 중단하고 EBS 볼륨은 보존

중지 상태에서도 EBS 볼륨 저장 요금은 계속 발생합니다.
컴퓨팅 요금만 0원이 되며, 재시작 시 공인 IP는 새로 할당됩니다.

Appendix

인스턴스 유형 선택 기준: t4g.nano와 t4g.micro

인스턴스 유형	vCPU · 메모리	단일 스프링부트 컨테이너 구동 관점
t4g.nano	2 vCPU · 0.5GiB	JVM 힙과 Docker 데몬, OS 프로세스가 경합해 OOM 종료 위험이 큼
t4g.micro	2 vCPU · 1GiB	단일 컨테이너 구동에 필요한 최소 메모리 버퍼 확보

🏠 시스템 상주 영역

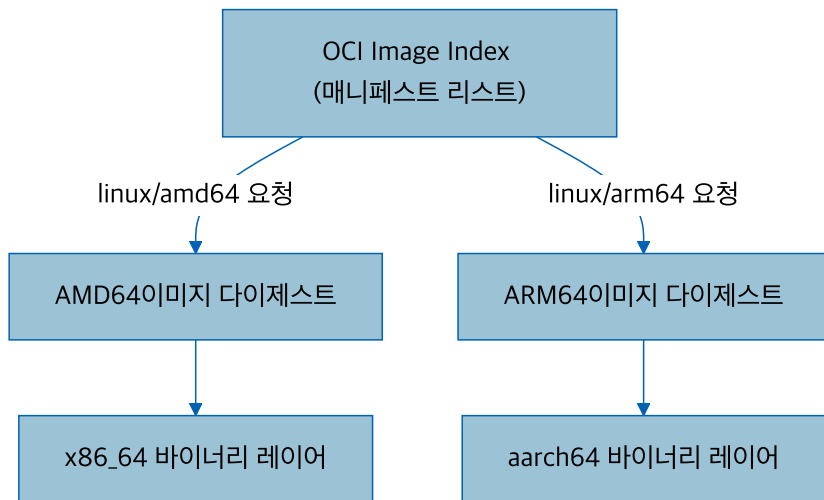
OS 커널과 Docker 데몬, `containerd`가 상시 메모리를 점유합니다

🚀 애플리케이션 영역

남은 용량 안에서 JVM 힙과 메타스페이스, 스레드 스택이 배분됩니다

JVM은 기본적으로 컨테이너에 할당된 메모리의 일정 비율을 최대 힙으로 잡습니다. 여기에 Docker 데몬과 `containerd`, 커널 캐시가 더해지므로 512MiB 환경에서는 커널 OOM 킬러가 스프링부트 프로세스를 강제 종료할 수 있습니다.

멀티 아키텍처 이미지 매니페스트 원리



📌 단일 태그, 복수 아키텍처

동일한 `:latest` 태그가 아키텍처별 다이제스트를 가리키는 인덱스 역할을 합니다

📁 선택적 다운로드

클라이언트 커널에 맞는 레이어만 내려받아 개발 환경과 배포 서버의 아키텍처 불일치를 흡수합니다

Docker 그룹 권한과 보안 고려사항

```
sudo usermod -aG docker $USER
newgrp docker
docker ps
```

항목	내용
편의성	<code>docker</code> 그룹에 사용자를 추가하면 <code>sudo</code> 없이 Docker 명령을 실행할 수 있습니다
위험	<code>/var/run/docker.sock</code> 접근 권한은 호스트 루트 파일시스템 마운트를 통해 사실상 <code>root</code> 권한과 동등합니다

다중 사용자가 접근하는 운영 서버에서는 그룹 추가를 신중히 제한하고, 필요한 계정에만 부여하거나 루트리스 모드를 검토합니다.

핵심 기술 면접 질문

공인 IP 회수 메커니즘

Elastic IP를 연결하지 않은 EC2 인스턴스를 재시작할 때 공인 IP가 변경되는 이유는 무엇인가요?

상태 저장 방화벽의 영향

AWS 보안 그룹이 상태 저장(Stateful) 방화벽인 점이 네트워크 운영에 미치는 영향은 무엇인가요?

아키텍처 불일치와 해결책

x86_64 기반 CI 러너에서 빌드한 이미지를 ARM64 EC2 인스턴스에서 실행할 때 발생하는 문제와 해결책은 무엇인가요?

컨테이너 재시작 정책

컨테이너 실행 시 `--restart unless-stopped` 옵션을 지정하는 기술적 이유는 무엇인가요?

모범 답변

공인 IP가 변경되는 이유

AWS는 한정된 IPv4 주소 자원을 효율적으로 운영하기 위해 인스턴스가 중지되면 할당했던 공인 IP를 회수해 동적 풀에 반환합니다. 이후 다시 시작할 때 풀에서 새 주소를 동적으로 부여하므로 IP가 바뀝니다. 고정 주소가 필요하다면 탄력적 IP를 생성해 인스턴스에 연결해야 합니다.

상태 저장 방화벽의 영향

인바운드로 진입한 정상 요청의 세션을 기억해 그에 대한 응답 아웃바운드 패킷을 자동으로 허용합니다. 따라서 8080 포트를 인바운드로 열었다면, 응답이 나가는 임시 포트에 대해 별도의 아웃바운드 규칙을 설정하지 않아도 통신이 유지됩니다.

모범 답변

아키텍처 불일치 문제와 해결책

CPU 명령어 세트가 일치하지 않아 `exec format error`가 발생하며 프로세스 기동이 거부됩니다. Docker Buildx와 QEMU에뮬레이터로 `linux/amd64`와 `linux/arm64`를 동시에 빌드하고, 두 아키텍처를 하나의 태그로 묶는 매니페스트 리스트 형태로 레지스트리에 푸시해 해결합니다.

`--restart unless-stopped`의 목적

커널 패치나 예기치 못한 호스트 재부팅으로 Docker 데몬이 다시 시작될 때, 관리자가 명시적으로 정지시키지 않았던 컨테이너를 자동으로 재기동하기 위함입니다. 일시적인 인스턴스 재부팅 이후에도 서비스 다운타임을 최소화할 수 있습니다.

깊이 알아보기

☁ 인스턴스 유형 변경

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/e-c2-instance-resize.html>

🔗 탄력적 IP

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/elastic-ip-addresses-eip.html>

📁 Ubuntu Docker 설치

<https://docs.docker.com/engine/install/ubuntu/>

🏗 멀티 플랫폼 빌드

<https://docs.docker.com/build/building/multi-platform/>

깊이 알아보기

🔌 컨테이너 재시작 정책

<https://docs.docker.com/engine/containers/start-containers-automatically/>

🔌 Docker 설치 후 설정

<https://docs.docker.com/engine/install/linux-postinstall/>



- EOF -